

STHYRA CRM · ENGINEERING

Frontend Engineering Playbook

A guide for the engineer building the web dashboard, the 360° viewer, the BIM viewer, and the mobile capture shell.

For: Frontend engineer
Working with: Senior Backend Engineer

v1.0 · 6-m
Build from

Sthyra CRM — Full-Stack Engineering Playbook

For: A full-stack engineer assigned to take Sthyra CRM from current state to Phase 1 MVP

Time to complete: ~10 weeks full-time for one engineer, ~4 weeks for a team of 3.

Read first: `README.md` (this repo), `STHYRA-SYSTEM-ARCHITECTURE.md` (AWS plan), `STHYRA-FEATURES-CATALOG.md` (every function), `STHYRA-DEV-FLOWCHARTS.md` (10 flows).

This playbook is organized as **9 sequential phases**. Each phase has:

- A clear **definition of done**
- **Concrete tasks** with file paths, code shapes, and test commands
- **Time estimate**
- **Verification checklist** you can run yourself

Use it as a sprint board. Tick tasks as you finish them.

§ Phase 0 — Onboarding (Day 1–2)

Definition of done: You can run all tests, boot all services, and read the working code confidently.

› Task 0.1 — Read the master plan and architecture

```
# These are committed alongside the code:
cat README.md
cat STHYRA-PHASE-0-REPORT.md           # 44 pages
cat STHYRA-SYSTEM-ARCHITECTURE.md      # 62 pages
cat STHYRA-FEATURES-CATALOG.md         # 9 pages
cat STHYRA-DEV-FLOWCHARTS.md           # 27 pages

# And the original synthesis:
cat ~/.hermes/plans/2026-08-08_090307-sthyra-crm-visual-intelligence-platform.md
```

Time: 2–3 hours. Don't skip this — every later decision depends on what you learn here.

› Task 0.2 — Set up the dev environment

```
# Prerequisites
node --version           # need 22.22.3+
pnpm --version           # need 11.x
docker --version         # for Postgres
git --version

# Clone + install
cd ~/projects/plumb
pnpm install

# Build
pnpm build

# Tests
pnpm test
# Expected: 114/114 pass across 8 packages
```

› Task 0.3 — Boot everything locally

```
# Terminal 1: Postgres
docker compose up -d postgres

# Terminal 2: org-service against Postgres
DATABASE_URL=postgres://plumb:plumb@localhost:5432/plumb \
  pnpm --filter=@sthyra-crm/org-service start:pg &

# Terminal 3: project-service
pnpm --filter=@sthyra-crm/project-service start:inmem &

# Terminal 4: user-service
pnpm --filter=@sthyra-crm/user-service start:inmem &

# Terminal 5: membership-service
pnpm --filter=@sthyra-crm/membership-service start:inmem &

# Terminal 6: dashboard
pnpm --filter=@sthyra-crm/dashboard dev

# Smoke test
curl http://localhost:8080/v1/health # → 200 {"status":"ok"}
curl http://localhost:8082/v1/health
open http://localhost:3000 # dashboard
```

› Task 0.4 — Read the working code in this order

1. `packages/tokens/src/index.ts` — design tokens
2. `packages/observability/src/index.ts` — request-id plugin
3. `packages/auth/src/index.ts` — auth middleware
4. `services/org-service/src/index.ts` — Repository pattern, Region union
5. `services/org-service/src/postgres-repo.ts` — parameterized SQL pattern
6. `services/org-service/src/http.ts` — RFC 7807, Idempotency-Key
7. `services/membership-service/src/http.ts` — `@sthyra-crm/auth` integration
8. `apps/dashboard/src/lib/api.ts` — server-side fetch + request-id

Verification checklist:

- [] `pnpm test` shows 114 passing tests
- [] `pnpm build` builds all packages
- [] All four services boot and respond to `/v1/health`
- [] Dashboard renders at localhost:3000

§ Phase 4 — Three.js 360° Viewer (Week 4–5)

Definition of done: The dashboard renders a 360° equirectangular image, pan/zoom works at 60fps on M1, fallback to WebGL2 works.

› Task 4.1 — Add three.js + @react-three/fiber to dashboard

Files to modify:

- `apps/dashboard/package.json` — add `three`, `@react-three/fiber`, `@react-three/drei`
- `apps/dashboard/src/components/viewer-360/index.tsx` — new component

› Task 4.2 — Equirectangular camera

```
import { Canvas, useTexture } from '@react-three/fiber';
import { Sphere } from '@react-three/drei';

function Equirect({ url }: { url: string }) {
  const tex = useTexture(url);
  return (
    <Sphere scale={[-1, 1, 1]}>
      <meshBasicMaterial map={tex} side={1} />
    </Sphere>
  );
}
```

› Task 4.3 — Camera controls + LOD

Use drei's `OrbitControls` or write custom. Add:

- WASD pan, Q/E roll, arrows pitch/yaw (keyboard map per dev flowchart §9)
- Touch pan/pinch
- FPS budget: 60fps M1, 45fps Intel Iris Xe
- LOD downshift on low-end GPU via WebGL renderer info

› Task 4.4 — `/projects/[id]/capture/[captureId]` page

Wire up the route. Fetch capture from capture-service. Load equirectangular texture from pre-signed S3 URL. Mount the viewer full-screen.

Verification checklist:

- [] Renders 8K equirectangular at 60fps on M1
- [] Falls back to WebGL2 on WebGPU-unsupported browsers
- [] `prefers-reduced-motion` disables auto-camera
- [] FPS budget enforced; LOD kicks in on low-end GPU
- [] Snapshot of viewer is `< 800ms` to first pixel

§ Phase 5 — Field Notes & Issues Mobile Shell (Week 5–6)

Definition of done: A field user on iOS/Android can create an issue with a photo, offline, and it syncs when connectivity returns.

› Task 5.1 — Initialize mobile KMM module

```
# Create the KMM module
cd apps
mkdir mobile-kmm
cd mobile-kmm
# Use the KMM plugin in Android Studio, or initialize manually
```

› Task 5.2 — Sync engine (KMM shared logic)

File: `apps/mobile-kmm/src/commonMain/kotlin/com/plumb/sync/SyncEngine.kt`

Pattern from `docs/dev-flowcharts/dev-03-offline-sync.mmd` :

- LocalID = UUIDv7
- Lamport timestamps
- Idempotent upload via clientId + contentHash
- Pull-by-cursor

› Task 5.3 — iOS shell

File: `apps/mobile-ios/CaptureView.swift`

Use SwiftUI + SwiftData (or SQLite via GRDB). Capture photo, tag floor/area, attach to issue, queue for sync.

› Task 5.4 — Android shell

File: `apps/mobile-android/CaptureActivity.kt`

Use Compose + Room. Mirror the iOS flow.

› Task 5.5 — E2E test: airplane mode → reconnect

Use Maestro or XCUITest. Steps:

1. Open app, create issue with photo, no network
2. Verify SQLite has the issue with `LocalID`, status `pending_sync`
3. Toggle Wi-Fi on
4. Wait for sync (verify `LocalID → ServerID` mapping)
5. Verify server has the issue

Verification checklist:

- [] Offline issue creation persists across app kill
- [] Sync on reconnect with no duplicates
- [] Photo upload works with chunked resumable

- [] iOS XCUITest + Android Espresso both green
-

§ Phase 6 — BIM Model Viewer + IFC Ingest (Week 6–7)

Definition of done: A VDC engineer can upload an IFC file, see it in the 3D viewer, click an element to see its BIM GUID.

› Task 6.1 — BIM service skeleton

```
services/bim-service/  
├─ src/  
│  ├─ index.ts  
│  ├─ postgres-repo.ts  
│  ├─ http.ts           # POST /v1/projects/:id/bim (upload IFC, parse, tessellate)  
│  ├─ tessellate.ts     # IFC → glTF conversion  
│  ├─ cli.ts  
│  └─ *.test.ts  
└─ ...
```

Use `web-ifc` or `IFC.js` for IFC parsing. Tessellate to glTF on upload, store in S3 `sthyra-crm-bim-{region}/`.

› Task 6.2 — BIM viewer component

File: `apps/dashboard/src/components/bim-viewer/index.tsx`

- three.js scene
- glTF loader
- Click handler → returns `{ GUID, elementName, position }`
- Section plane, measurement (distance, angle, area)
- Clash highlight (initial: color BIM elements red where another model overlaps)

› Task 6.3 — BCF 3.0 import/export

Files: `services/bim-service/src/bcf.ts`

Implement BCF 3.0 XML format. Round-trip with external tools (Solibri, BIMcollab).

Verification checklist:

- [] IFC upload + parse < 60 s for 100 MB file
- [] Viewer renders 10K elements at 60fps
- [] Click on element returns GUID
- [] BCF topic creation from a click works
- [] Tests: ≥15 for bim-service

§ Continuous: ship code every day

› Daily workflow

```
# 1. Pull latest
git pull
pnpm install

# 2. Make a branch
git checkout -b feat/copilot-streaming

# 3. Write the failing test FIRST
vim services/copilot-service/src/streaming.test.ts
pnpm --filter=@sthyra-crm/copilot-service test
# Expect: red

# 4. Implement
vim services/copilot-service/src/streaming.ts

# 5. Run tests
pnpm --filter=@sthyra-crm/copilot-service test
# Expect: green

# 6. Typecheck and lint
pnpm typecheck
pnpm lint

# 7. Commit with conventional format
git add -A
git commit -m "feat(copilot): streaming SSE for tool-call responses"

# 8. Push and open PR
git push -u origin feat/copilot-streaming
```

› Per-PR checks

The CI pipeline runs:

1. Lint (ESLint + Prettier)
2. Typecheck (tsc)
3. Unit tests (vitest / node:test)
4. Integration tests (with real Postgres)
5. Build (tsc / next build)
6. SBOM + signed image
7. Auto-deploy to ephemeral preview env (4h TTL)

› Per-RC checks

The release-candidate pipeline adds:

- SLO burn rate check
- Lighthouse CI bundle budgets
- Coverage $\geq$ 80% on changed lines

- Security scan + SBOM diff
 - Accessibility audit
 - DR drill within 30 days
-

§ Architecture decisions to NOT re-decide

These are pinned in the master plan. Don't second-guess:

Decision	Why
Tenancy data-modeled (every record carries <code>region</code>)	Compliance invariant
Repository pattern (InMemory for tests, Postgres for prod)	Standard testing seam
REST + RFC 7807 + Idempotency-Key at the edge	Pinned in §9
Request-ID via <code>@sthyra-crm/observability</code>	Every log has <code>request_id</code>
Strict TypeScript (<code>noUncheckedIndexedAccess</code> , etc.)	Real bugs caught during dev
Teal+amber palette (not cyan+copper)	Field-safety: amber reserved for warnings
pnpm workspaces (not Yarn/npm)	Already set up; switching costs weeks
monorepo layout: <code>packages/</code> , <code>services/</code> , <code>apps/</code> *	Don't add new top-level dirs

§ Risk register (per master plan §15)

Risk	Mitigation
Field-network realism	Device field-test program; Network Link Conditioner in CI
GPU/driver matrix	GPU-fingerprint → known-issue registry
AI hallucination drift	Monthly blind-set refresh; quarterly third-party audit
PII leakage in test data	Nightly PII scan across all test buckets
Multi-region consistency	Monthly failover game day per region
Mobile camera hardware variability	Pin firmware in QA; test staging on N-1 firmware
3DGS generalization on HDR	Auto-fallback to NeRF; per-scene quality gates
Drone regulatory	LAANC at flight-planning; on-staff Part 107 advisor
FedRAMP Moderate timeline (12-18 mo)	Start in P2; StateRAMP as faster interim
Switching cost (OpenSpace imports)	Import tool GA-day-one with documented fidelity

§ Where to get help

- Stuck on a service pattern? Look at `services/org-service/` — it's the reference impl
 - Stuck on Postgres SQL? Mirror `services/org-service/src/postgres-repo.ts`
 - Stuck on tests? Mirror `services/org-service/src/postgres-repo.test.ts`
 - Stuck on auth? Look at `packages/auth/src/index.ts` and `services/membership-service/src/http.ts`
 - Stuck on design tokens? Look at `packages/tokens/src/index.ts`
-

§ Phase 1 MVP exit criteria (master plan §13)

- ☐ TTF capture < 5 min
 - ☐ 5 paid conversions (design partners)
 - ☐ NPS > 30
 - ☐ 114 tests still green
 - ☐ ≥ 80% coverage on changed lines
 - ☐ All SLOs green for 7 consecutive days
 - ☐ SOC 2 Type II evidence collection started
-

§ Total time budget

Phase	Scope	Time
0	Onboarding	2 days
1	Postgres parity	1 week
2	Real auth	1 week
3	Capture service	2 weeks
4	360 viewer	1 week
5	Mobile shell	2 weeks
6	BIM viewer	1 week
7	AI Copilot	2 weeks
8	Integrations	1 week
9	Deploy to AWS	1 week

For 3 engineers in parallel: ~4–5 weeks.

End of playbook.